

Komentované části kódu

```
//Načtení všech potřebných knihoven
#include <WiFi.h>
#include <ESPAsyncWebServer.h>
#include <ArduinoOTA.h>
#include <HTTPClient.h>
#include <cbuf.h>
#include <SPI.h>
#include <Arduino.h>
#include <Preferences.h>
#include <web_content.h>
#include <LiquidCrystal_I2C.h>
#include <IRremoteESP8266.h>
#include <IRrecv.h>
#include <BluetoothA2DPSink.h>
#include <Adafruit_VS1053.h>
#include <VS1053.h>
#include <RotaryEncoder.h>
```

Prvně je potřeba načíst veškeré knihovny potřebné pro funkci rádia. Přidány jsou knihovny Wifi.h, HTTPClient.h pro správnou funkci spojení se streamovacím serverem. Pro funkci vytvoření webového serveru přímo v ESP jsem použil knihovny ESPAsyncWebServer.h a mnou vytvořenou knihovnu web_content.h pro definování struktury HTML kódu.

Dále jsou zde knihovny jako ArduinoOTA.h pro funkci OTA nebo Preferences.h pro možnost ukládání proměnných natrvalo do NVS (Non-Volatile Storage), která je náhradou paměti EEPROM (Electric Erasable Programmable Read Only Memory) u Arduina.

Knihovny používané pro Bluetooth, jako jsou BluetoothA2DPSink.h a cbuf.h.

Knihovny SPI.h, VS1053.h a Adafruit_VS1053.h pro zvukovou kartu a knihovny pro periferie a ovládací prvky IRemoteESP8266.h, IRrecv.h, RotaryEncoder.h a LiquidCrystal_I2C.

```
//Definování pinů

//Zvuková karta VS1053b
#define VS1053_CS 5
#define VS1053_DCS 16
#define VS1053_DREQ 4
#define VS1053_CSSD 17
#define CLK 18
#define MISO 19
#define MOSI 23

//Infračervený přijímač (OUT)
#define IR_REC 2

//Tlačítko pro přepínání mezi WiFi a Bluetooth režimy
#define BT_PIN 33

//Rotační enkódér
#define BUTTON_PIN 12
#define DT_PIN 13
#define CLK_PIN 14

//Název zařízení v Bluetooth režimu
#define BT_NAME "ESP32 RADIO"

//Velikost bufferu (vyrovnávací paměti) pro knihovnu Adafruit_VS1053.h
- režim Bluetooth
#define BUFSIZE 32
```

Další úryvek kódu definuje přiřazení pinů jednotlivým proměnným.

V příloze práce v souboru main.cpp je vyzobrazena inicializace knihoven i vytvoření potřebných proměnných na řádcích 48 - 194.

```

//Funkce pro spuštění serveru v případě, že se ESP32 nedokázalo př
    ipojit k žádné WiFi
void serverRun() {
    WiFi.softAP(server_ssid, server_heslo);

    server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
        request->send(200, "text/html", obsahWeb); //Vytvoří se server
        na IP zobrazené na displeji, kde se zobrazí html kód z prom
        ěnné obsahWeb, která je uložena ve flash spolu s programem
    });
    server.on("/", HTTP_POST, [](AsyncWebServerRequest *request){
        if(request->hasParam("ssid", true) && request->hasParam("
            password", true)) {
            AsyncWebParameter* ssidParam = request->getParam("ssid",
                true);
            AsyncWebParameter* passwordParam = request->getParam("
                password", true);

            String newSSID = ssidParam->value();
            String newPassword = passwordParam->value();
            //Ukládání zadaného SSID a hesla do sharedPrefs
            inicializované při spuštění ESP
            preferences.begin("wifi", false);
            preferences.putString("ssid", newSSID);
            preferences.putString("password", newPassword);
            preferences.end();
            delay(1000);
            ESP.restart();
        } else {
            request->send(400, "text/plain", "Chybí potřebné parametry"
                );
        }
    });
    server.begin();
    //Serial.println("Server běží!");
}

```

Funkce pro vytvoření webového serveru na ESP32. Webový serveru bude mít strukturu souboru web_content.h, kterou si můžete zobrazit v příloze práce.

```
if (server_vych_mod == RADIO){
    serverRun();

    Serial.println("Přepínám na režim SERVER");
    server_vych_mod = SERVER;
}

else {
    serverRun();
}

}

else if (WiFi.isConnected()){
    Serial.println("Spojení s WiFi úspěšně navázáno");
}
```

Spouští se při neúspěšném připojení pomocí zadaných parametrů SSID a hesla k WiFi. Při tomto je třeba se připojit na ESP32 pomocí WiFi a zadat do webového prohlížeče IP adresu zobrazenou na LCD. Poté potvrdit odeslání nového SSID a hesla.

```
//Funkce volána při přepnutí z WiFi režimu do Bluetooth režimu
void startBluetooth(){
    Serial.println("Přepínám na BT režim");
    if (rad_vych_mod != BT_REZIM){
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("BLUETOOTH");
        lcd.setCursor(0,1);
        lcd.print("Nastavovani..."); //Vypsání BLUETOOTH na displej
    }

    stream_prehravac.stopSong(); //Pozastavení inicializované knihovny
    VS1053.h
    stream_prehravac.softReset();
    delay(1000);
    if (!bt_prehravac.begin()) {
        Serial.println(F("Chyba VS1053b, zkontroluj zapojení pinů!"));
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print("Chyba E1");
        while(1);
    }
}
```

```

}

bt_prehravac.begin(); //Start zvukové karty pomocí knihovny
    Adafruit_VS1053.h

    ArduinoOTA.end(); //Vypnutí OTA
    WiFi.mode(WIFI_OFF); //Vypnutí WiFi
    WiFi.disconnect();
    Serial.println("WiFi režim: " + WiFi.getMode()); //Testovací výpis
        pro kontrolu
    delay(1000);
    circBuffer.flush(); //Vyprázdnění vyrovnávací paměti
    Serial.println(stream_prehravac.isChipConnected()); //Testovací vý
        pis pro kontrolu
    a2dp_sink.set_stream_reader(dat_stream, false); //Nastavení
        Bluetooth streamu
    a2dp_sink.set_avrc_metadata_callback(metadata); //Nastavení metadat
        - nepoužívám jej pro výpis jinde, než v konzoli, není třeba
    a2dp_sink.start(BT_NAME); //Start Bluetooth se jménem v BT_NAME
    Serial.println(a2dp_sink.get_connected_source_name()); //Testovací v
        ýpis pro kontrolu
    delay(1000);
    circBuffer.write((char *)bt_wav_header, 44);
    delay(1000);

    Serial.println("Bluetooth režim"); //Testovací výpis pro kontrolu

    if (rad_vych_mod != BT_REZIM){ //Výpis informace o režimu na LCD
        displej spolu s názvem BT
        lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("BLUETOOTH");
    lcd.setCursor(0,1);
    lcd.print(BT_NAME);
    }

    rad_vych_mod = BT_REZIM; //Přepnutí režimu na BT_REZIM
}

```

Pomocí funkce výše se pak zapíná Bluetooth režim (respektive se vypíná WiFi a zapíná Bluetooth), kdy je třeba vypnout WiFi z důvodu interference ve stejném kmitočtovém pásmu. Poté je třeba vypnout inicializovanou zvukovou kartu knihovny **VS1053.h** a inicializovat zvukovou kartu knihovny **Adafruit_VS1053.h**.

```
//Tato funkce je volána při přepínání z Bluetooth do rádiového režimu
void stopBluetooth(){
    lcd.clear(); //Výpis toho, co se s rádiem děje
    lcd.setCursor(0, 0);
    lcd.print("BLUETOOTH");
    lcd.setCursor(0,1);
    lcd.print("Vypinani...");
    a2dp_sink.end();
    delay(500);

    Serial.println("Přepínám na WiFi režim");
    bt_prehravac.stopPlaying();
    bt_prehravac.reset();
    ESP.restart();
}
```

Funkce pro vypnutí Bluetooth a přepnutí do WiFi režimu. Kvůli nemožnosti správně vypnout WiFi v ESP jsem nucen restartovat rádio a přepojit do WiFi režimu až po startu.

```
void Enkoder() {
    static int poz = 1;
    int novaPoz = enkoder->getPosition(); //Získání pozice
    int smerST;
```

V této funkci enkóderu se nachází zvlášť switch pro přepínání režimů **WiFi/Bluetooth** a v každém z nich poté ještě switch mezi přepínáním **hlasitost/stanice**.

```
case WIFI_REZIM:
if (poz != novaPoz) {
    switch (enk_vych_mod) { //Volba režimu enkóderu
        case enkoder_stanice: //Přepínání stanic
            //Zjištění směru otáčení
            smerST = (int)(enkoder->getDirection());
            //Podmínky pro otáčení enkóderem
            if (smerST > 0 && ID_stanice < pocet_stanic) {
                ID_stanice++;
            } else if (smerST < 0 && ID_stanice > 1) {
                ID_stanice--;
            }
        }
    }
}
```

```

Serial.print("pozST:"); //Vypisování do serialu (pro testování)
Serial.print(ID_stanice);
Serial.print("smerST:");
Serial.println(smerST);
poz = novaPoz; //Uložení pozice do poz
zaloha_stanice = ID_stanice; //Uložení ID stanice do
    zaloha_stanice
break;
case enkoder_hlasitost: //Změna hlasitosti
    int smerVO = (int)(enkoder->getDirection()); //smerVO nabývá
        hodnot 1 a -1 dle knihovny RotaryEncoder.h
    if (smerVO > 0 && Volume < 100) { //Pokud je 1, zvýšení
        hlasitosti o 1
        Volume++;
    } else if (smerVO < 0 && Volume > 29) { //Pokud je -1, snížení
        hlasitosti o 1
        Volume--;
    }
    Serial.print("pozVO: "); //Vypisování do serialu (pro testování)
    Serial.print(enkoderPozVO);
    Serial.print("smerVO: ");
    Serial.println(smerVO);
    Serial.print("Volume: ");
    Serial.print(Volume);
    poz = novaPoz; //Zápis nové polohy
    break;
}
}
break;

```

Zde je zase vidět, jakým způsobem se chovají přepnuté case ve WiFi režimu. Vše je na základě směru otáčení enkóderu. Samozřejmě nechybí ošetření hraničních intervalů hlasitosti.

```

case BT_REZIM:

if (poz != novaPoz) {
    switch (enk_vych_mod) {
        case enkoder_stanice: //Využil jsem již existující
            enkoder_stanice (přepíná skladby v playlistu připojeného zaří
                zení)
            //Zjištění směru otáčení

```

```

    smerST = (int)(enkoder->getDirection());
    if (smerST > 0) {
        a2dp_sink.next(); //Přepnutí vpřed
    } else if (smerST < 0) {
        a2dp_sink.previous(); //Přepnutí vzad
    }
    Serial.print("posST:"); //Vypisování do serialu (pro testování)
    Serial.print(" dirST:");
    Serial.println(smerST);
    Serial.print(bluetooth_media_title);
    poz = novaPoz; //Zápis nové polohy
    break;

case enkoder_hlasitost: //Jako u WiFi režimu je zde změna
    hlasitosti
    //Zjištění směru otáčení
    int smerVO = (int)(enkoder->getDirection());
    //Podmínky pro otáčení enkódérem
    if (smerVO > 0 && VolumeBT > 1) {
        VolumeBT--;
    } else if (smerVO < 0 && VolumeBT < 101) {
        VolumeBT++;
    }
    Serial.print("posVO:"); //Vypisování do serialu (pro testování)
    Serial.print(enkoderPozVO);
    Serial.print(" dirVO:");
    Serial.println(smerVO);
    Serial.print(" VolumeBT:");
    Serial.print(VolumeBT);
    poz = novaPoz; //Zápis nové polohy
    break;
}
}
break;
}

```

Opět přepínání **hlasitost/stanice** v Bluetooth režimu.

//Funkce pro tlačítko, které přepíná režimy enkóderu

```
void handleButton() {  
    if (digitalRead(BUTTON_PIN) == LOW) {  
        if (!buttonPressed) {  
            buttonPressed = true;  
  
            //přepnutí stavu  
            if (enk_vych_mod == enkoder_stanice) {  
                enk_vych_mod = enkoder_hlasitost;  
                Serial.println("Měním hlasitost!");  
            } else {  
                enk_vych_mod = enkoder_stanice;  
                Serial.println("Přepínám stanice!");  
            }  
        }  
    } else {  
        buttonPressed = false;  
    }  
}
```

Funkce sloužící pro podchycení funkčnosti tlačítka na enkóderu.

Mění stavy přepnutí **hlasitost/stanice** v proměnné **enk_vych_mod**.

```
void IR0vladac() {  
    if (irrecv.decode(&decod_prij)) {  
        Serial.println(decod_prij.value, HEX);  
    }
```

V tomto úryvku nalezneme funkci IR diody. Příchozí signál je třeba dekodovat a následně na základě přijatého hexadecimálního kódu (HEX) určit proces, který se má vykonat.

```

switch (rad_vych_mod){ //Pro WiFi a Bluetooth rozlišné funkce

    case WIFI_REZIM:

        if (decod_prij.value == 0xFFB04F) { // tlačítko "#"

            Serial.println("Restart za 2s!");
            delay(1500);
            ESP.restart();
        }

```

Jak jsem zmínil výše, je třeba přiřadit hexadecimálnímu kódu jeho úlohu. Zde je příklad HEX kódu **0xFFB04F** přiřazeno pro restart ESP (jedná se o tlačítko se symbolem #). Tyto stejné úlohy lze přiřazovat v Bluetooth režimu zvlášť.

```

String Scroll_displeje(String LCD_text){
    String LCD_hot;
    String LCD_proces = odsazeni + LCD_text + odsazeni;
    LCD_hot = LCD_proces.substring(x1,x2);
    x1++;
    x2++;
    if (x1>LCD_proces.length()){
        x1=16;
        x2=0;
    }
    return LCD_hot;
}

```

Zde vidíme funkci pro možnost pohyblivého displeje (respektive jeho prvního řádku) z důvodu délky názvu stanic, země původu stanice a identifikátoru stanice. Kód je inspirován kódem ze stránky s názvem I2C LCD interfacing with Arduino Display Scrolling Text and Custom Characters. [?]

```
//Funkce pro tlačítko, které přepíná mezi režimy WiFi a BT
void BTbutton() {
    //Ošetření nechtěného přepínání
    BTbuttonState = digitalRead(BT_PIN);
    unsigned long currentTime = millis();
    if (BTbuttonState == LOW) {
        if (currentTime - BTlastButtonPressTime >= BTdebounceDelay) {
            delay(50);

            if (rad_vych_mod == WIFI_REZIM) {
                //Zapnutí Bluetooth, pokud jsme ve WiFi režimu
                startBluetooth();
                Serial.println("Přepínám do BT režimu");
                delay(1000);
                Serial.println("Přepnuto!");
            } else if (rad_vych_mod == BT_REZIM) {
                //Zastavení Bluetooth, pokud jsme v Bluetooth režimu
                stopBluetooth();
                Serial.println("Přepínám do WiFi režimu");
                delay(1000);
                Serial.println("Přepnuto!");
            }
            BTlastButtonPressTime = currentTime;
        }
    }
}
```

Tlačítko pro přepínání režimů **WiFi/Bluetooth**. Je zde nastavena ochrana proti nechtěnému delšímu držení tlačítka.

```

//Funkce pro samotný stream
void stream(){
    if(!webclient.connected() || pamet_ID_stanice!=ID_stanice){

        if(webclient.connect(host, 80)){                                //Př
            ipojení se k požadovanému streamu
            webclient.print(String("GET ") + cesta+ " HTTP/1.1\r\n" + "Host:
                " + host + "\r\n" + "Connection: close\r\n\r\n");
            pamet_ID_stanice=ID_stanice;
            Serial.println(nazev); //Vypsání názvu do serialu, pro testovací
                účely
            lcd.clear(); //čistka displeje
        }

        else {
            Serial.println("Nepodařilo se připojit k serveru.");
            lcd.clear();
            lcd.setCursor(0,0);
            lcd.println("Chyba E2");
            stream_prehravac.stopSong();
        }
    }

    if(webclient.available() > 0){
        uint8_t bytesread = webclient.read(mp3buff,32);                //naplnění
            bufferu při dostupnosti serveru
        stream_prehravac.playChunk(mp3buff,bytesread);                //přehrávání
            audio streamu z bufferu na výstup VS1053
    }
}

```

Zde je proces samotného žádání streamu po streamovacím serveru. V případě úspěšného připojení se naplní vyrovnávací paměť (buffer) a přehraje audio obsah.

```
//Naplnění bufferu
void dat_stream(const uint8_t *data, uint32_t length) {
    if (circBuffer.room() > length) {
        if (length > 0) {
            circBuffer.write((char *)data, length);
        }
    } else {
        Serial.println("\nŽádná streamovací data!");
    }
}
```

Naplnění dat do **circBuffer**.

```
//Průběh Bluetooth streamu, volá se ve smyčce
void handle_stream() {
    if (circBuffer.available()) {
        if (bt_prehravac.readyForData()) { //Dotaz na zvukovou kartu, zda
            může přijímat další data
            int bytesRead = circBuffer.read((char *)mp3buff, BUFFSIZE);
            //Zachycení chybného čtení dat
            if (bytesRead != BUFFSIZE) Serial.printf("Málo dat v bufferu,
                kvalita audia může být ovlivněna", bytesRead);
            bt_prehravac.playData(mp3buff, bytesRead); //Zaslání dat do
                zvukové karty
        }
    }
}
```

Hlavní funkce pro běh bluetooth spojení. Data z **circBuffer** se použít na zvukové desce.

```

void setup ()
{
  Serial.begin(115200);
  delay(500);
  SPI.begin(); //Inicializace SPI
  if (!bt_prehravac.begin()) {
    Serial.println(F("Nedetekováno žádné VS1053B, zkontroluj zapojení
    pinů! Zařízení nebude fungovat!"));
    while(1);
  }
  stream_prehravac.begin(); // Start zvukové karty VS1053B pomocí
    knihovny VS1053.h

  pinMode(BUTTON_PIN, INPUT_PULLUP); //Nastavení GPIO pinů jako vstup (
    tlačítko na enkóderu a Bluetooth tlačítko)
  pinMode(BT_PIN, INPUT_PULLUP);

  Serial.println("Enkódér úspěšně inicializován!");
  enkoder = new RotaryEncoder(DT_PIN, CLK_PIN, RotaryEncoder::LatchMode
    ::TWO03); //Inicializace rotačního enkóderu
  irrecv.enableIRIn(); //Inicializace IR diody
  attachInterrupt(digitalPinToInterrupt(DT_PIN), getPoz, CHANGE); //
    Nastavení přerušení při změně na DT_PIN nebo CLK_PIN
  attachInterrupt(digitalPinToInterrupt(CLK_PIN), getPoz, CHANGE);

  lcd.init(); // Inicializace LCD displeje
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Radio - SIL0074");

  lcd.setCursor(0,1);
  lcd.print("Start radia");

  Serial.println("Startuji");

  WiFi.mode(WIFI_STA); //Nastavení režimu WiFi (STA - station mode)

```

```
delay(1000);  
// WiFi.begin(ssid, pass);  
//serverRun();  
WiFi.begin(ssid, pass); //Připojení pomocí SSID a hesla
```

Úryvek kódu je z funkce **setup()**, kde se provede nastavení režimů pinů, režimy enkóderu, aktivace displeje nebo se nastaví režim WiFi na STA (Station mode). Je zde taktéž pokus o prvotní připojení na WiFi s parametry SSID a heslem.

```
if (!WiFi.isConnected()) { //Jestliže se nepodaří ani to, informuje  
    se uživatele o chybě spojení na serial  
Serial.println("Chyba připojení!");  
  
//Spuštění serveru a přepnutí do režimu SERVER  
if (server_vych_mod == RADIO){  
    serverRun();  
  
Serial.println("Přepínám na režim SERVER");  
server_vych_mod = SERVER;  
}  
  
else {  
    serverRun();  
}  
}  
  
else if (WiFi.isConnected()){  
    Serial.println("Spojení s WiFi úspěšně navázáno");  
}
```

Podmínka pro úspěšné či neúspěšné připojení.

V případě úspěšného spojení přejde rádio volně setup funkcí bez změny režimu. Jestliže se však nepodaří připojit, spustí se server režim, ve kterém lze manuálně nastavit SSID a heslo.

```
//Zprovoznění OTA
ArduinoOTA.setHostname("ESP32");
//ArduinoOTA.setPassword("SIL0074"); //možnost nastavení hesla
ArduinoOTA.begin();
```

Spuštění OTA, je zde možnost nastavit heslo, tuto možnost jsem nevyužil.

```
preferences.begin("radio", true); //Získání hodnot z sharedPrefs
Volume = preferences.getInt("Volume", Volume);
VolumeBT = preferences.getInt("VolumeBT", VolumeBT);
ID_stanice = preferences.getInt("ID_stanice", ID_stanice);
preferences.end();
```

Načtení dat z **preferences**, do kterých se ukládá aktuální hodnota hlasitosti z obou režimů a identifikátor stanice. Jsou tak zachované i po restartu rádia.

```
//Hlavní smyčka programu
void loop()
{
switch(server_vych_mod){//Přepínání mezi RADIO a SERVER režimem
  case RADIO:
```

Ukázka hlavní smyčky, kde se pomocí switche **server_vych_mod** najede do příslušného režimu **radio/server**.

```
switch(rad_vych_mod){//Přepínání mezi WiFi a Bluetooth režimem

{
case WIFI_REZIM:
```

V režimu **radio** se poté přepíná mezi režimy **WiFi/Bluetooth**.

```

//V případě server režimu
case SERVER:

    if (!IPdone) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Server rezim");//Výpis na LCD displej
        lcd.setCursor(0, 1);
        lcd.print("IP: "); //Výpis na LCD displej
        String serverIP = WiFi.softAPIP().toString();
        lcd.print(serverIP); //Výpis IP serveru na LCD displej
        Serial.println("IP VYPSANA");
        IPdone = true;
    }

    Prikazy(); //Detekce příkazů v server režimu
    break;

```

V režimu **server** se už nepřepíná a pouze se vypíše na LCD displej potřebný text pro informaci uživatele.

```

case WIFI_REZIM:

    if (Obnova >= 450){ //Po kolika cyklech se má obnovit LCD

        Obnova = 0; //Reset

        String nazevCely = nazev + "|" + nar + "|" + ID_stanice;
        lcd.setCursor(0, 0);
        String pohybText = Scroll_displeje(nazevCely);
        lcd.print(pohybText); //vypsání infa o stanici na první řádek LCD
    }

```

Zde se celý režim odehrává pod podmínkou počítadla **Obnova**, určuje, po kolika opakováních se vykoná aktualizace hlasitosti a LCD displeje. Hodnota 450 je ideální pro plynulou obnovu a minimalizaci efektu **ghosting** na displeji.

Dále se zde volá funkce pro pohyblivý název stanice, zemi původu stanice a identifikátor stanice.

```

if (Volume != predVolume) {
    lcd.setCursor(0, 1);
    lcd.print("                "); //Smazání druhého řádku LCD
    displeje
    lcd.setCursor(0, 1);
    if (Volume < 30){
        lcd.print("Zvuk vypnut");//vypsání na druhý řádek LCD
    }
    else{
        lcd.print("Hlasitost: "); //vypsání na druhý řádek LCD
        lcd.print(mapVolume);
        lcd.print ("%");
    }
    //aktualizace hlasitosti
    predVolume = Volume;
}

else{
    lcd.setCursor(0, 1);
    if (Volume < 30){
        lcd.print("Zvuk vypnut"); //vypsání na druhý řádek LCD
    }
    else{
        lcd.print("Hlasitost: "); //vypsání na druhý řádek LCD
        lcd.print(mapVolume);
        lcd.print ("%");
    }
}

preferences.begin("radio", false); //ukládání do prefs
preferences.putInt("Volume", Volume);
preferences.putInt("ID_stanice", ID_stanice);
preferences.end();
}

```

Funkčnost a aktualizace hlasitosti ve smyčce pouze však ve chvíli, kdy se hlasitost změnila (aktualizace neproběhne, jestliže je hlasitost na stejné úrovni). Uložení hodnot **Volume** a **ID_stanice** do **preferences**.

```

switch(ID_stanice){           // Přepínání stanic - ID stanice
    case 1:

        nazev = "Hitradio Orion";
        host = "ice.abradio.cz";
        cesta= "/orion128.mp3";
        nar = "CZ";
        break;
    case 2:

        nazev = "Evropa 2";
        host= "ice.active.net";
        cesta= "/fm-evropa2-128";
        nar = "CZ";
        break;

```

Ukázka funkčnosti různých stanic, nachází se zde switch pro přepínání **ID_stanice**. Každá stanice má pak vlastní case a parametry, které se načítají a zobrazují na displej.

Já mám nastaveno 30 stanic, tudíž mám 30 case.

```

stream(); //funkce pro stream

IROvladac(); //aktualizace IR

Enkoder(); //aktualizace enkóderu

handleButton(); //aktualizace tlačítka

Prikazy(); //aktualizace příkazů

BTbutton(); //aktualizace BT tlačítka

```

Opět volány všechny funkce zmíněné výše.

```

//Funkce v BT režimu
case BT_REZIM:
    //Pokud není BT spuštěno, zapni jej
    if (!a2dp_sink.is_connected()){
        startBluetooth();
    }

    //Refresh LCD v BT režimu
    if (Obnova >= 20000){
        Obnova=0;
        bt_prehravac.setVolume(VolumeBT, VolumeBT); //Nastavení hlasitosti
        VS1053b

```

V bluetooth režimu se vyskytuje počítadlo **Obnova** taktéž, je nastaveno na rozdílnou hodnotu, protože je bluetooth na výkon o dost náročnější a při častějších obnovách mohou nastat problémy. Taková vysoká hodnota počítadla však nijak nezhoršuje zážitek z rádia.

Taktéž se nastavuje hlasitost na zvukovou desku (v této knihovně žáleží na levém a pravém kanálu, tudíž je třeba vyplnit dvě pozice).

```

//Mapování hodnot hlasitosti v jiném pořadí a v jiném intervalu
int mapVolumeBT = map(VolumeBT, 1, 100, 100, 1);
if (mapVolumeBT == 0 || VolumeBT == 254){
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("BLUETOOTH");//Výpis na LCD displej
    lcd.setCursor(0,1);
    lcd.print("Zvuk vypnut");//Výpis na LCD displej
}

```

Úryvek kódu ukazuje mapování hodnot v opačném pořadí, knihovna totiž pracuje s opačnými hodnotami a chceme, abychom na LCD displeji viděli, že 100% je opravdu hlasitost maximální, nikoliv minimální.

Pokud je zvuk vypnut, na displej se vypíše příslušný text.

```

preferences.begin("radio", false); // Uložení hodnot do shared prefs
preferences.putInt("VolumeBT", VolumeBT);
preferences.end();

```

Uložení hodnoty hlasitosti v bluetooth režimu **VolumeBT** do **preferences**.

```
handle_stream(); //aktualizace streamu
```

```
IROvladac(); //aktualizace IR
```

```
Enkoder(); //aktualizace enkóderu
```

```
handleButton(); //aktualizace tlačítka
```

```
Prikazy(); //aktualizace příkázů
```

```
BTbutton(); //aktualizace BT tlačítka
```

```
Obnova++;
```

Volání potřebných funkcí a inkrementace proměnné **Obnova**.